



Мифы и легенды о разработке через тестирование

Чурсин Никита





Привет!

Я Никита Чурсин

Старший разработчик
в Озон Банке



- В разработке с 2013
- В Озон Банке в команде KYC (знай своего клиента)
- Пишу код через тесты и люблю работать в парах
- Заставил немецких банкиров писать авто-тесты
- Член программного комитета DUMP секции «Тестирование»
- Играем с женой дуэтом: фортепьяно и ударные

Познакомимся?

Напиши в чат, из какого ты
подразделения и сколько лет
в Озоне



Сегодня разберем

01 Что такое TDD?

02 Как проблемы можно
решить с помощью TDD?



03 Мифы и правда о TDD

Организационные моменты

1

Работаем 60 минут,
перерыва не будет

2

Задаем вопросы

3

Фиксируем ключевые
мысли

4

Презентация будет,
запись – попробуем.





Тесты — хорошо или
плохо?



Можете ли вы
деплоить код
опираясь только на
работу тестов?

Типичные проблемы тестов (после кода)

- Не покрыты все кейсы
- Причина падения неочевидна
- Код сложно тестировать
- Мы любим свой код 

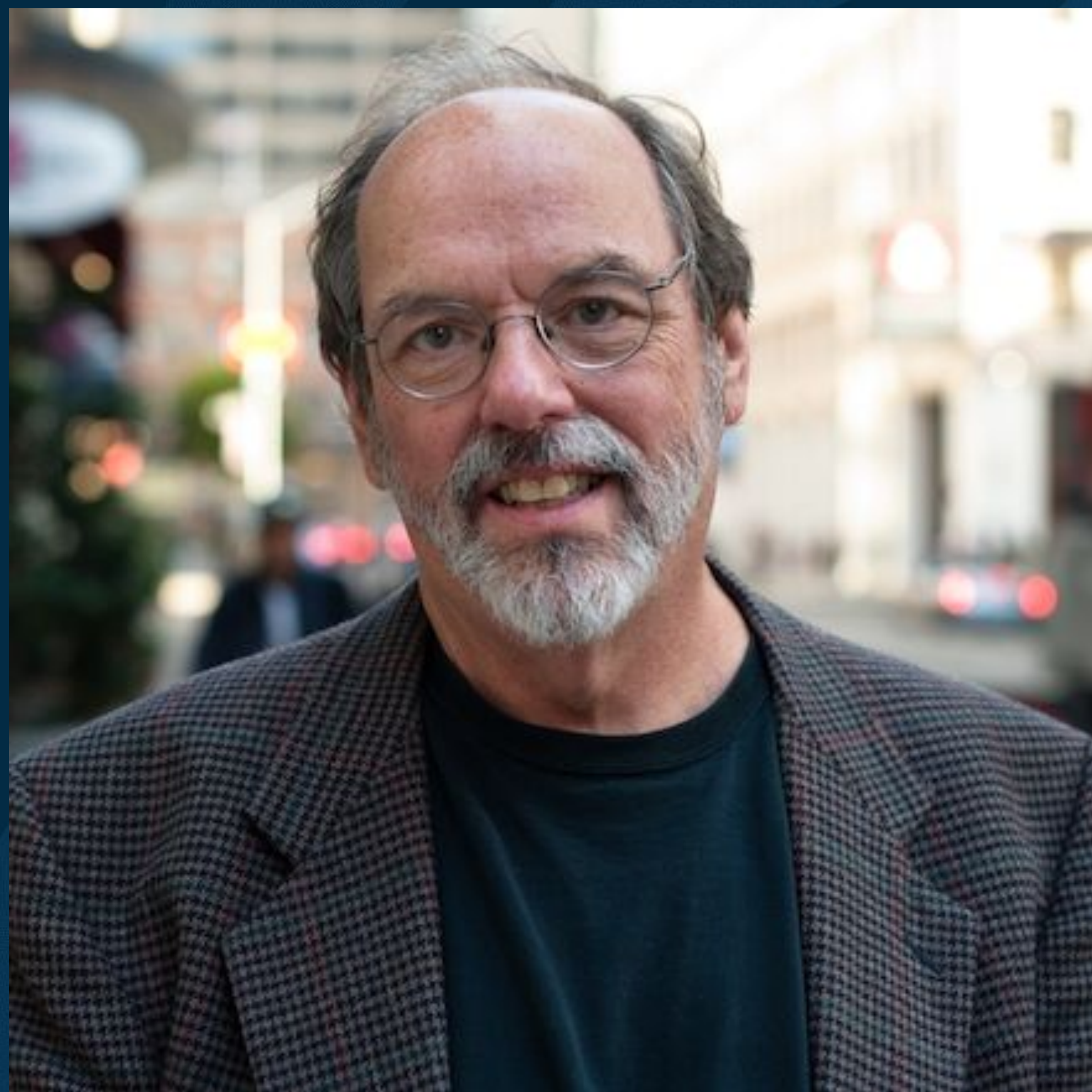
Как должно быть

- ~~Не покрыты все кейсы~~ Тесты покрывают максимум кейсов
- ~~Причина падения неочевидна~~ Тесты чётко показывают причину падения
- ~~Код сложно тестировать~~ Код легко тестируется
- Мы любим свой код 





Немного истории



Уорд
Каннингем



Кент Бек

Идея TDD — экстремальное тестирование

- Тесты — хорошо
- Каждый тест должен упасть
- Минимум необходимого кода
- Фокусируемся на одной вещи за раз
- Удаление дублирования

Что даёт TDD?

- Бесстрашный рефакторинг
- Доверие к тестам
- Доверие к коллегам
- Документация
- Чистый код, который работает



Но, я слышал, TDD —
это сложно

Миф: писать через тесты — сложно

- Это долго
- Это дорого и я просто не смогу это продать
- Надо написать все тесты сразу
- «Я не могу думать о тестах, ведь надо думать о коде»

С чего начинается разработка?



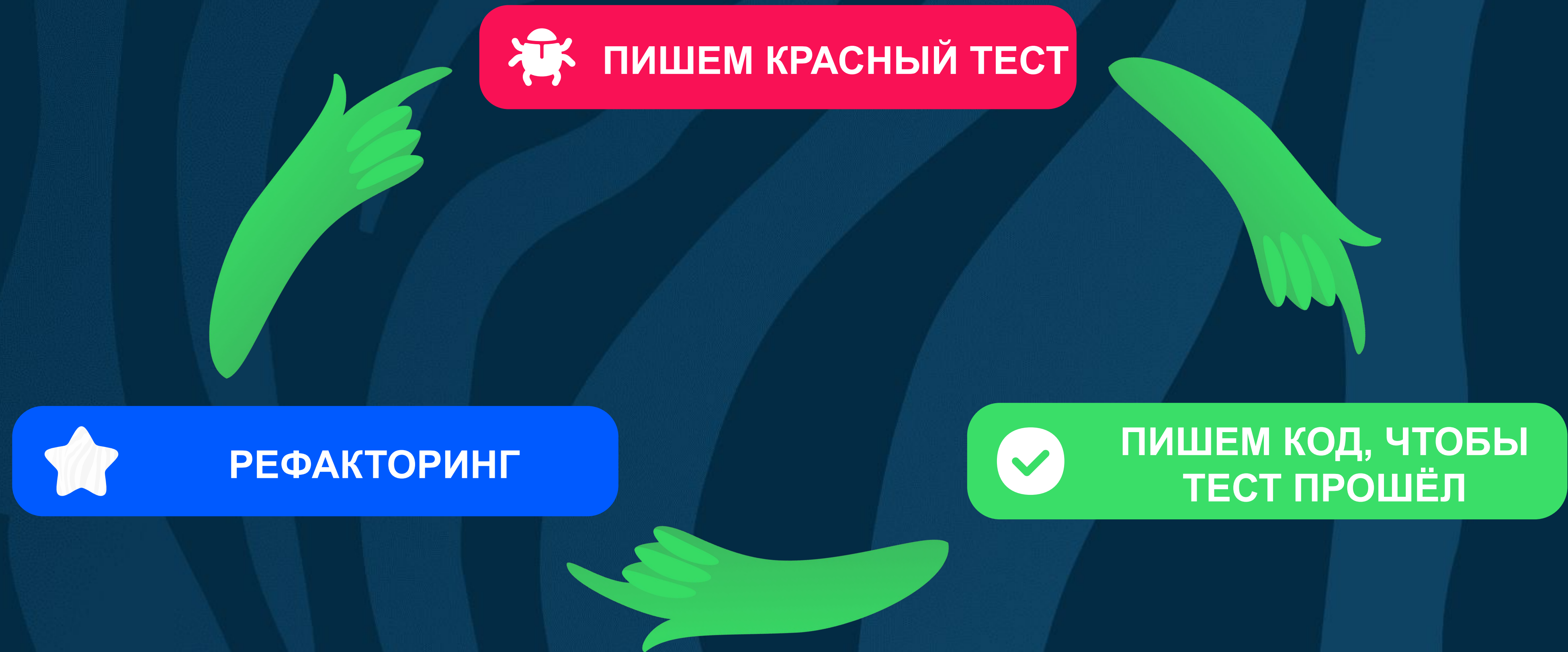
Разработка
начинается с
требований



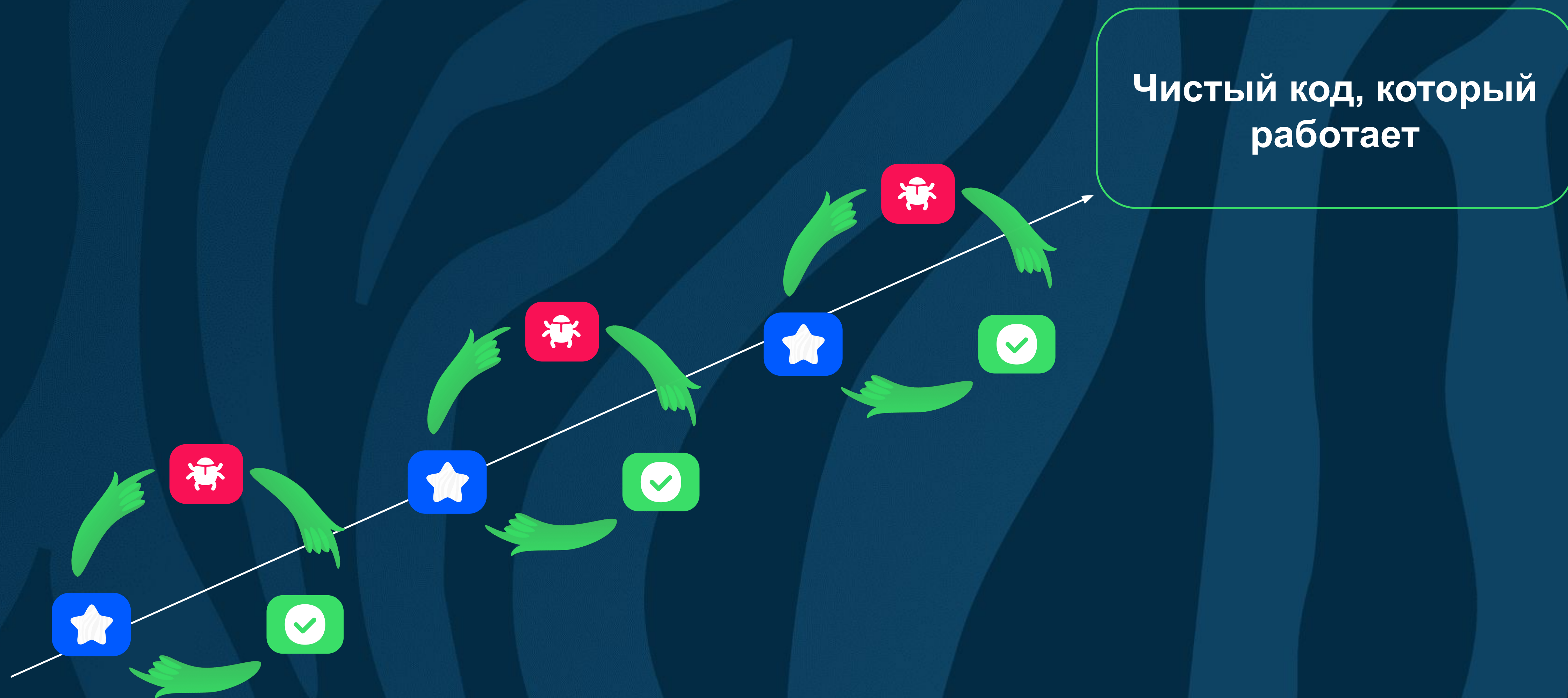
Думать о том, как
проверить код, ты
должен



Цикл TDD



Разработка через тесты



Бизнес понимает только в деньгах

- Время — деньги
- TDD увеличивает время разработки на 15-35% [1, 2]
- TDD уменьшает количество дефектов на 18-90% [1,2,3]
- Исследования отмечают более медленную деградацию кода на долгом промежутке времени [4] => более предсказуемую скорость поставки

1. [Realizing Quality Improvement Through Test Driven Development: Results and Experiences of Four Industrial Teams](#), 2008
2. [An Initial Investigation of Test Driven Development in Industry](#), 2003
3. [Test-Driven Development as a Defect-Reduction Practice](#), 2003
4. [A Longitudinal Study of the Test-driven Development Practice in Industry](#), 2007



Но такие тесты
сложны в поддержке

Миф: тесты сложно поддерживать

- Куча моков
- Хрупкие тесты
- Требуют вовлечения всей команды
- Работает только на проектах, которые уже разработаны через тесты
- Тесты нельзя удалять



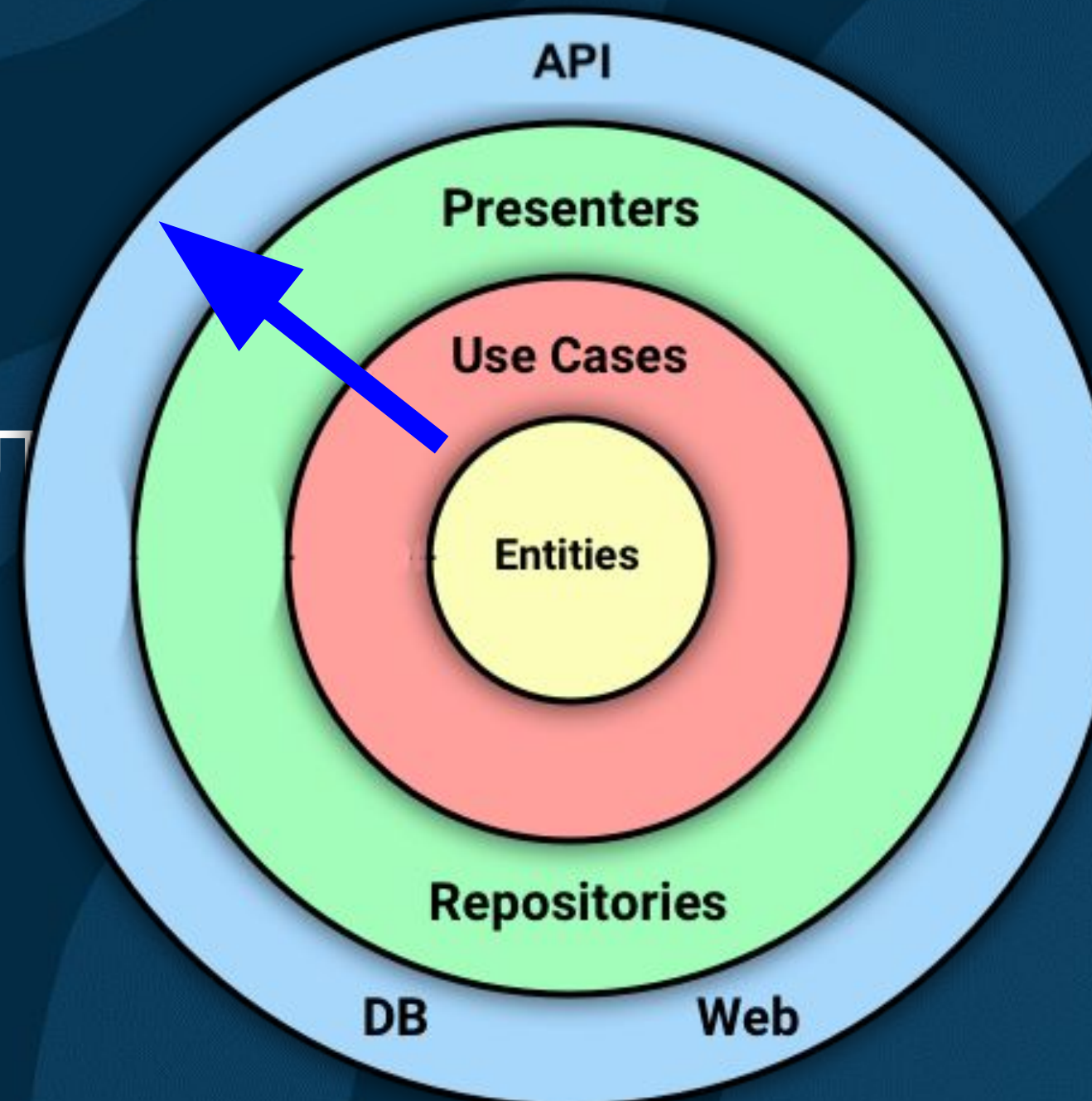
London
Mockist
Outside-In

VS

Chicago
Classic
Inside-Out

Изнутри наружу (Chicago)

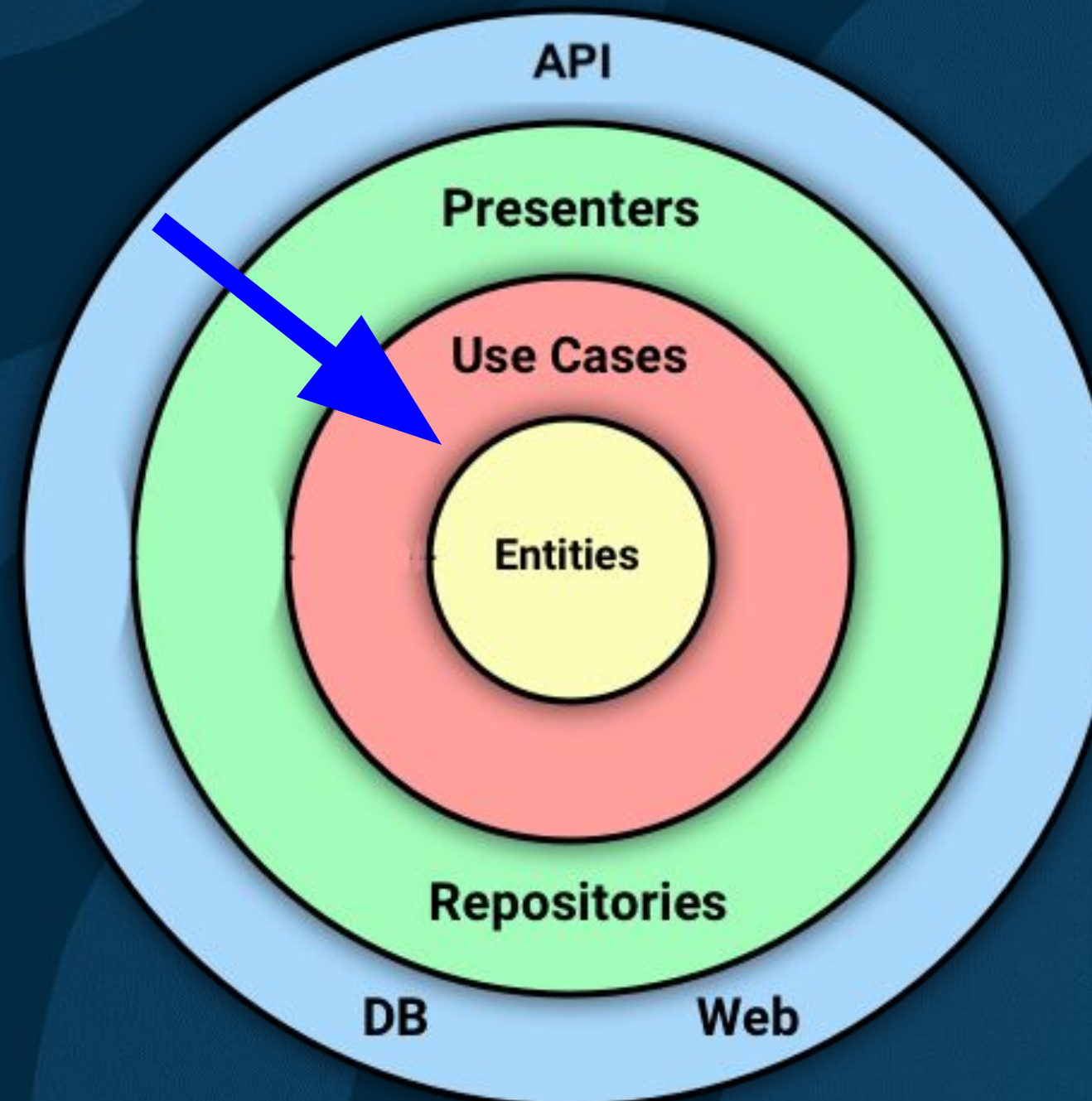
- Начинаем изнутри (доменная область)
- Поднимаемся вверх
- Наращиваем функционал параллельно с тестами



- + Тесты отвязаны от деталей реализации
- + Высокое зацепление (cohesion)
- + Минимум моков
- Можно наоверинжинирить

Снаружи внутрь (London)

- Начинаем снаружи (API)
- Фокус на поведении
- Продвигаемся вниз
- Мокаем зависимости



- + Фокус на поведении
- + Чёткая изоляция
- Хрупкие тесты
- Привязка к реализации => сложный рефакторинг

Вовлечение команды

- Это классно!
- Но это обязательно
- И вам придётся следить за тестами на ревью внимательнее

Работает только на проектах с TDD

Не работает в легаси

Легаси — это код, который страшно менять.

- Фиксим баги через тесты
- Approval-тесты
- Покрываем тестами для уверенности
- «Тупой» рефакторинг с покрытием тестам
- TCR

TCR: test - commit || revert

- На каждое изменение запускаем тесты
- Если тесты зелёные — код коммитится
- Если тесты красные — код удаляется

TSR: область применения

- Рефакторинг на всегда зелёных тестах
- Исследование легаси кода

Тесты нельзя удалять

Льзя*

* Если они дублируют другие тесты



TDD это про утопию

Миф: TDD — это про идеальный мир

- В реальной жизни фичи устроены немного сложнее, чем «функция X должна принять имя и вывести приветствие с этим именем»
- «Тесты написанные после модуля не идут в мусорку»
- Изначальные требования могут быть поняты или сформулированы неверно
- Это только про юнит-тесты, а для интеграционных и выше TDD неприменимо
- TDD — шляпа, а вот BDD — топчик

Фичи — это сложно

- Учебные примеры для TDD — это обучающие примеры
- IBM разрабатывала драйвера через тесты в рамках исследования[1]:
 - для части кассового аппарата — ящика с деньгами
 - сканнера SKU
 - магнитного считывателя
 - клавиатуры
 - принтеры

1. [A Longitudinal Study of the Test-driven Development Practice in Industry](#), 2007

Требования меняются

- TDD заставляет думать о требованиях: начинаем с составления списка тестов

Подумай о требованиях

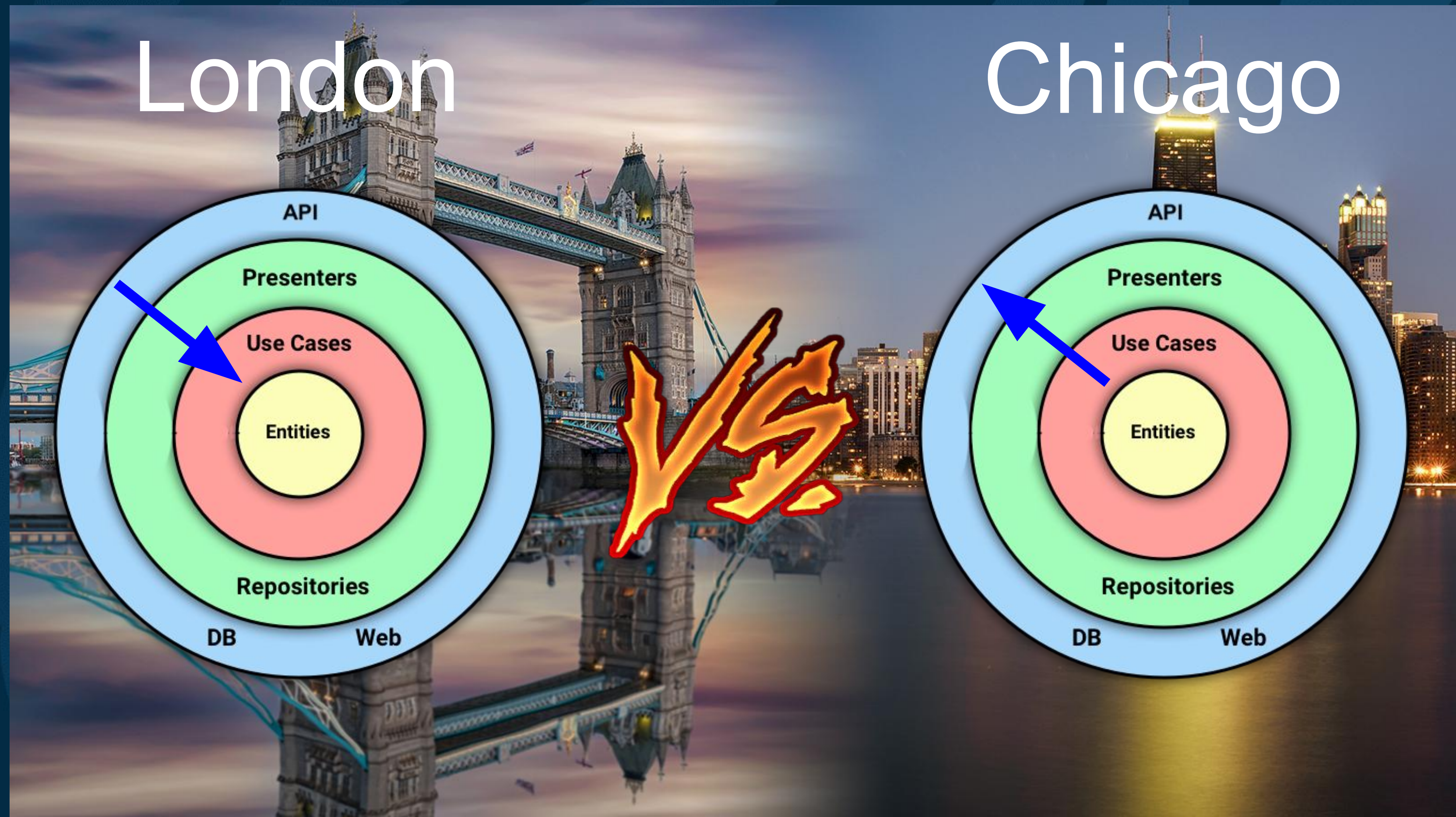
Компания	Число акций	Цена	Всего
IBM	1000	25 USD	25 000 USD
Novartis	400	150 CHF	60 000 CHF
		Итого	65 000 USD

- $5 \text{ USD} + 10 \text{ CHF} = 10 \text{ USD}$, если курс обмена 2:1
- $5 \text{ USD} * 2 = 10 \text{ USD}$

Требования меняются

- TDD заставляет думать о требованиях: начинаем с составления списка тестов
- Раньше возникают вопросы
- Больше уточнений на ранних этапах — меньше переделывать

Юнит- и интеграционные тесты



TDD vs BDD



vs



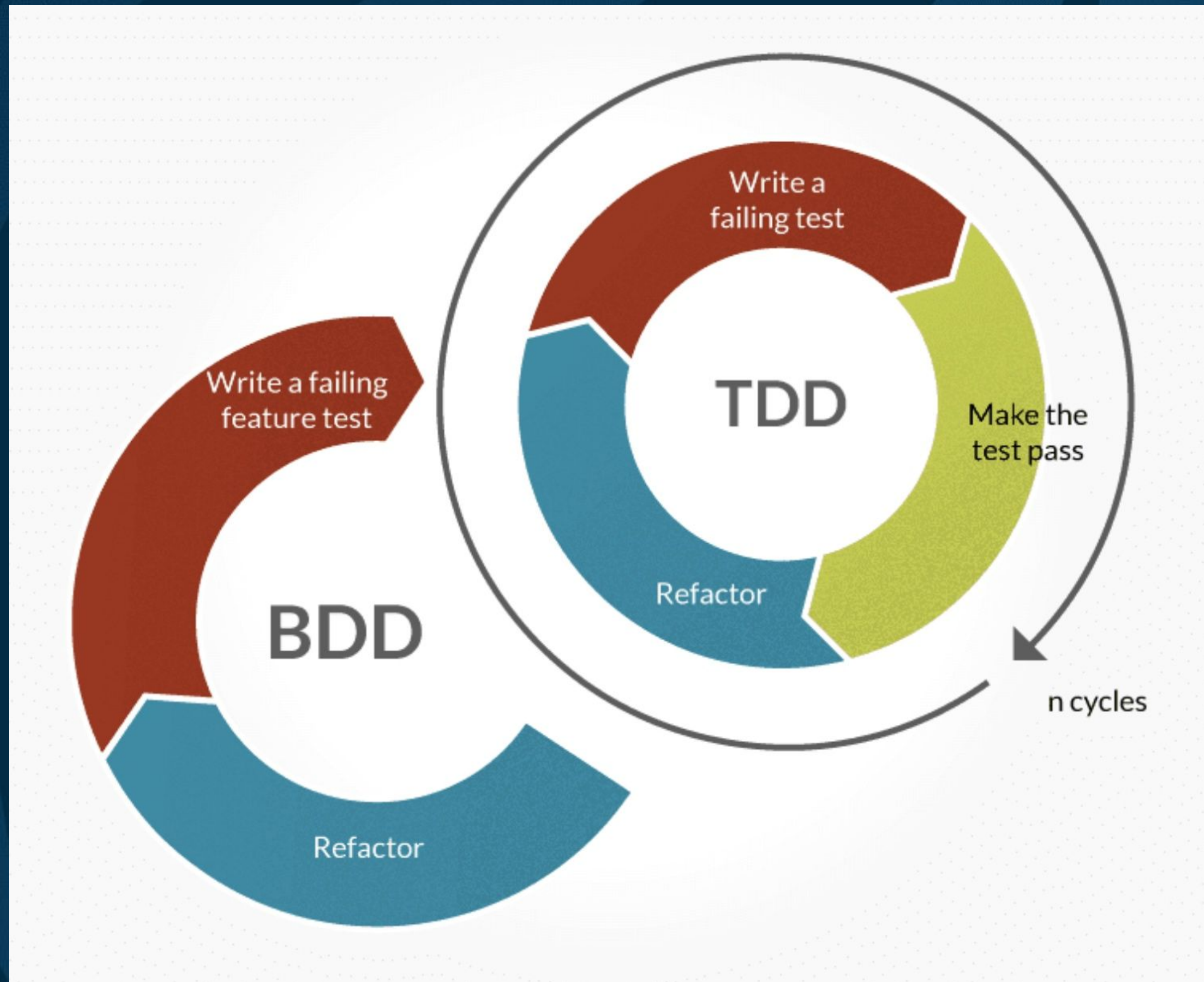
Что такое BDD?

- Это отдельная история...
- Behavior Driven Development
- Способ общения о требованиях на примерах
- На выходе получаем сценарии
- В чистом виде — это не про автоматизацию тестирования
- И вообще не про тестирование

Но есть ATDD

- Acceptance Test Driven Development
- Автоматизация сценариев BDD

TDD feat BDD



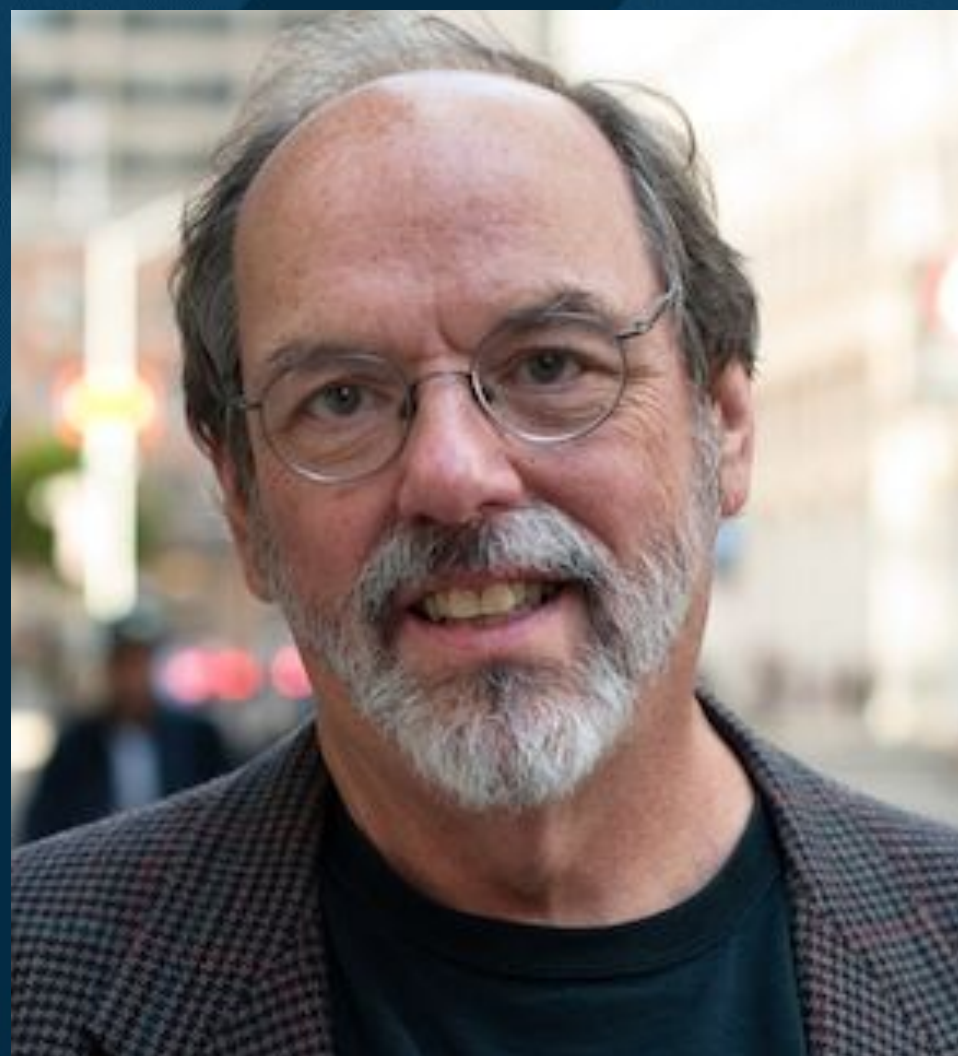


TDD — серебряная
пуля!

Миф: TDD — это серебряная пуля

- TDD = нет багов
- TDD = 100% покрытия
- TDD заменяет дизайн
- Волосы будут длинными и шелковистыми

Но это не точно



Уорд Каннингем



Кент Бек

TDD — не волшебная палочка

- Тесты хороши ровно настолько, насколько их придумал разработчик
- С TDD вы контролируете скорость — и если вам не надо 100% покрытия, его не будет
- Дизайн никто не отменял



Что в итоге?

В итоге

- TDD — техника разработки, в которой мы перед добавлением функционала пишем падающий тест
- Писать все тесты сразу — не надо!
- Анализируй требования
- Иди маленькими шагами
- Пиши чистый код, который работает — и рефачь без страха!

Сообщество в Ozon

Присоединяйтесь к каналу ~go-tdd
в Chatzone!



Используй силу, Люк



Если тест **красным** ты не
видел — не верь ему!



Воркшоп по TDD

